1 2 9 0

UNIVERSIDADE D
COIMBRA

Redes de Comunicação – Licenciatura de Engenharia Informática

Protocolo “PowerUDP”

Realizado por:

Francisco Gouveia – 2023214517

Miguel Pinheiro – 2023217656

Índice

Arquitetura da Solução 3
Componente Servidor 3
Gestão de Clientes: 3
Autenticação: 3
Gestão de Configuração: 3
Notificação de Encerramento: 3
Componente Cliente 3
Comunicação com o Servidor: 3
Comunicação entre Clientes: 3
Receção de Configurações: 3
Funcionalidades do Protocolo PowerUDP 4
1. Registo e Inicialização 4
2. Envio de Mensagens 4
3. Receção de Mensagens (receiver_loop no cliente) 4
4. Retransmissão de Mensagens 5
5. Geração de ACK/NACK 6
6. Ordenação de Mensagens 6
7. Gestão de Configuração 6
8. Encerramento 7
Opções Arquiteturais 7
Threads vs. Forks: 7

Arquitetura da Solução

Componente Servidor

Gestão de Clientes:

O servidor aceita conexões TCP de clientes para o seu registo. Cada cliente conectado é gerido por uma thread dedicada (handle_client), permitindo o tratamento concorrente de múltiplos clientes.

Autenticação:

O servidor autentica os clientes durante o processo de registo, verificando uma chave pré-compartilhada.

Gestão de Configuração:

O servidor mantém a configuração ativa do protocolo PowerUDP. Ele pode receber pedidos de alteração de configuração de clientes (via TCP) e, subsequentemente, dissemina a nova configuração para todos os clientes registados utilizando mensagens multicast.

Notificação de Encerramento:

O servidor pode notificar todos os clientes sobre o seu encerramento iminente através de uma mensagem multicast específica (MSG_TYPE_SERVER_SHUTDOWN).

Componente Cliente

Comunicação com o Servidor:

O cliente comunica com o servidor utilizando TCP para fins de registo e para solicitar alterações na configuração do protocolo. O registo é autenticado através de uma chave pré-compartilhada (PSK).

Comunicação entre Clientes:

A troca de mensagens de dados entre clientes é realizada utilizando o protocolo PowerUDP sobre sockets UDP.

Receção de Configurações:

O cliente recebe atualizações de configuração do servidor através de mensagens multicast.

Funcionalidades do Protocolo PowerUDP

1. Registo e Inicialização

- Cliente: A função init_protocol estabelece uma conexão TCP com o servidor, envia uma mensagem de registo contendo a PSK, e aguarda uma confirmação. Se o registo for bem-sucedido, o cliente recebe a configuração inicial do protocolo do servidor. São também inicializados os sockets UDP (para comunicação com outros clientes) e multicast (para receber configurações do servidor). São criadas as threads receiver_loop e display_loop.

- Servidor: A thread handle_client recebe a mensagem de registo do cliente, valida a PSK. Se válida, envia uma confirmação "OK" e a configuração atual do protocolo ao cliente. O socket do cliente é adicionado a uma lista de clientes ativos.

2. Envio de Mensagens

- A função send_message é responsável por encapsular a mensagem da aplicação num PowerUDPPacket, adicionando o cabeçalho PowerUDP com o tipo MSG_TYPE_DATA e um número de sequência incremental (next_seq_num_udp).

- A decisão de implementar ou não mecanismos de confiabilidade depende da configuração atual (current_config).

3. Receção de Mensagens (receiver_loop no cliente)

- Esta thread escuta nos sockets UDP e multicast.
- Pacotes de Dados (UDP):
    - Ao receber um pacote de dados (MSG_TYPE_DATA), verifica se a funcionalidade de sequenciação está ativa.
    - Validação de Números de Sequência: Se o sequenciamento (current_config.enable_sequence) estiver ativo, o cliente mantém um estado (peer_seq_state_pool) para cada peer do qual recebe mensagens, armazenando o último número de sequência processado. Se um pacote chega com um número de sequência inferior ou igual ao

último processado (e não é o primeiro pacote, seq_num 0), é considerado fora de ordem ou repetido. Neste caso, o pacote é descartado e um NACK é enviado ao remetente. Caso contrário, o número de sequência é atualizado e um ACK é enviado.

    - Se o pacote for válido e processado, a sua carga útil é colocada numa fila (message_queue) para ser consumida pela thread display_loop e posteriormente pela aplicação através da função receive_message.

- Pacotes ACK/NACK (UDP):
    - Ao receber um ACK ou NACK, a thread receiver_loop verifica se corresponde a algum pacote de dados enviado anteriormente e que está a aguardar confirmação (gerido pelo ack_wait_pool).
    - Se encontrar uma correspondência, sinaliza a thread que efetuou o send_message (que está em espera numa variável de condição), indicando o estado do ACK/NACK.

- Pacotes Multicast:
    - Se receber uma mensagem de atualização de configuração (MSG_TYPE_DATA contendo ConfigMessage), atualiza a current_config local.
    - Se receber uma mensagem de encerramento do servidor (MSG_TYPE_SERVER_SHUTDOWN), inicia o processo de encerramento do cliente.

4. Retransmissão de Mensagens

- Esta funcionalidade é ativada pela flag current_config.enable_retransmission.
- Quando ativa, após o envio de um pacote de dados, o cliente entra num ciclo de espera por um ACK ou NACK.
- Temporizador: É implementado usando pthread_cond_timedwait. Um slot no ack_wait_pool é reservado para a mensagem enviada, e a thread de envio espera numa variável de condição associada a esse slot.
- Cálculo do Timeout: O valor do timeout para cada tentativa é calculado com base no current_config.base_timeout.
    - Exponential Backoff: Se current_config.enable_backoff estiver ativo, o timeout é duplicado a cada tentativa falhada (até ao current_config.max_retries), conforme a fórmula Tn = Tmin * 2^n. A função calculate_timeout implementa esta lógica.

5. Geração de ACK/NACK

- Quando um cliente recebe um pacote de dados (MSG_TYPE_DATA):
    - Se o pacote for processado com sucesso (e a validação de sequência, se ativa, passar), um pacote PowerUDPPacket com header.type = MSG_TYPE_ACK e o mesmo header.seq_num do pacote de dados recebido é enviado de volta ao remetente.
    - Se a validação do número de sequência falhar (pacote duplicado ou fora de ordem), um pacote PowerUDPPacket com header.type = MSG_TYPE_NACK e o header.seq_num do pacote de dados problemático é enviado de volta ao remetente.

6. Ordenação de Mensagens

Quando a opção enable_sequence está activada em current_config, o cliente passa a verificar a sequência de cada pacote recebido. Para tal, mantém uma estrutura peer_seq_state_pool que assenta, para cada remetente conhecido, no último número de sequência processado (last_seq_num_processed). Sempre que chega um pacote de dados multicast, o cliente extrai o seu número de sequência (received_seq_num_udp) e compara-o com o valor armazenado em peer_seq_state_pool para aquele remetente. Se received_seq_num_udp for inferior ou igual a last_seq_num_processed (à excepção do primeiro pacote, que pode ter número 0), considera-se que o pacote está duplicado ou chegou fora de ordem: nesse caso, ele é simplesmente descartado e é enviado um NACK ao remetente. Caso contrário, o pacote é aceite, last_seq_num_processed é actualizado para received_seq_num_udp e é enviado um ACK confirmando o processamento bem-sucedido.

7. Gestão de Configuração

No cliente, a função request_protocol_config envia ao servidor, através de uma ligação TCP, um pacote cujo campo ClientServerTCPHeader.message_type está definido como TCP_MSG_TYPE_CONFIG_REQUEST e cujo conteúdo (payload) é a nova configuração pretendida (ConfigMessage).

No servidor, a thread responsável pelo atendimento de cada cliente (handle_client) deteta este tipo de mensagens e procede à atualização da variável global current_config, protegida por um mutex para garantir a consistência dos dados. De seguida, invoca broadcast_config_to_clients, que distribui a current_config actualizada a todos os clientes registados através de uma mensagem multicast. Esta mensagem é empacotada num PowerUDPPacket com o campo header.type definido como MSG_TYPE_DATA, tendo como payload o próprio ConfigMessage.

8. Encerramento

A função close_protocol_internal é invocada sempre que o cliente decide terminar sessão (por exemplo, ao receber o comando “exit”) ou quando recebe do servidor uma notificação de encerramento. Nesse momento, são sinalizadas as threads receiver_loop e display_loop para que concluam o seu trabalho, é enviada uma mensagem de desconexão TCP ao servidor (desde que o encerramento não tenha sido iniciado pelo próprio servidor), e em seguida procedem-se ao fecho de todos os sockets (TCP, UDP e multicast) e à libertação dos recursos associados.

No servidor, o tratamento do sinal SIGINT (por exemplo, quando se pressiona Ctrl+C) é assegurado pela função handle_sigint, que dá início ao processo de encerramento. Essa função invoca send_shutdown_to_all_clients, responsável por emitir um PowerUDPPacket com o campo header.type definido como MSG_TYPE_SERVER_SHUTDOWN através do canal multicast, de modo a notificar todos os clientes de que o serviço vai terminar. Por fim, são encerrados todos os sockets de cliente ativos, bem como o socket de escuta principal do servidor, concluindo assim o processo de desligamento.

Opções Arquiteturais

Threads vs. Forks:

Ambos os componentes, cliente e servidor, utilizam threads (pthreads) para gerir concorrência.

- No cliente, threads separadas são usadas para a interface do utilizador, receção de pacotes (UDP e multicast), e exibição de mensagens. Isto permite que o cliente permaneça responsivo enquanto realiza operações de rede em segundo plano.
- No servidor, uma nova thread é criada para cada cliente que se conecta, permitindo que o servidor lide com múltiplos clientes simultaneamente sem bloquear o loop principal de aceitação de novas conexões. Esta abordagem é geralmente mais leve em termos de recursos e partilha de memória em comparação com a utilização de processos separados (forks) para cada tarefa concorrente.